

ConvAI_Assignment_1_Group63_PS2

June 11, 2026

1 Problem Statement 2: Hybrid Retrieval for Financial Conversational Question Answering

BITS WILP — AI ML Assignment Submission

Conversational AI ASSIGNMENT GROUP 63

SL. NO	NAME	BITS ID
1	NEERADI CHANDRA SAGAR	2024AC05001
2	DEEPIKA ALAMURI	2024AC05631
3	SIDDHANT SINGH	2024AD05322
4	RAMPRASAD K	2024AC05628

Dataset: [FiQA-2018](#)

Runtime: Google Colab (CPU or GPU)

Embedding Models compared: all-MiniLM-L6-v2 vs all-MiniLM-L12-v2

Ranking Strategies compared: RRF vs Weighted Score Fusion

Similarity Metrics compared: Cosine vs Dot-Product

1.1 Table of Contents

1. Setup and Installation
 2. Module 1 — Data Preparation and Sparse Retrieval
 - Task 1: Financial Data Cleaning
 - Task 2: Sparse Search (BM25 + TF-IDF)
 3. Module 2 — Embeddings and Hybrid Retrieval
 - Task 3: Semantic Retrieval using Embeddings
 - Task 4: Hybrid Retrieval Pipeline
 4. Module 3 — Conversational QA and Evaluation
 - Task 5: Financial Conversational QA Pipeline
 - Task 6: Retrieval Evaluation and Analysis
 5. Technical Report
-

1. Setup and Installation

Run this cell first — installs all dependencies needed for the notebook.

```
[ ]: # Install dependencies
!pip install -q rank_bm25 sentence-transformers faiss-cpu datasets nltk
print('Done installing.')
```

```
0.0/588.9 kB
? eta -:--:--
583.7/588.9
kB 26.2 MB/s eta 0:00:01
588.9/588.9 kB
10.7 MB/s eta 0:00:00
0.0/18.2
MB ? eta -:--:--
4.1/18.2 MB 123.5 MB/s eta 0:00:01
10.4/18.2 MB 151.3 MB/s eta 0:00:01
17.9/18.2 MB
243.9 MB/s eta 0:00:01
18.2/18.2 MB
219.2 MB/s eta 0:00:01
18.2/18.2 MB
102.6 MB/s eta 0:00:00
Done installing.
```

```
[ ]: import warnings; warnings.filterwarnings('ignore')
import re, time, json, math
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import matplotlib.ticker as mticker
import seaborn as sns
from tqdm.auto import tqdm
from collections import defaultdict

import nltk
from nltk.tokenize import word_tokenize
from nltk.corpus import stopwords
from nltk.stem import PorterStemmer

from rank_bm25 import BM25Okapi
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity as sk_cosine
```

```

from sentence_transformers import SentenceTransformer
import faiss
from datasets import load_dataset

nltk.download('punkt', quiet=True)
nltk.download('punkt_tab', quiet=True)
nltk.download('stopwords', quiet=True)

print('All libraries imported successfully.')

```

All libraries imported successfully.

1.2 Task 1: Financial Data Cleaning and Preparation (1 Mark)

Dataset: FiQA-2018 — a financial question answering benchmark from Hugging Face.

Preprocessing steps applied: - Remove HTML tags and URLs - Expand finance-specific abbreviations (ETF, IPO, CAGR, RBI, etc.) - Lowercase and remove special characters - Stopword removal - Tokenization

```

[ ]: # Load Dataset
print('Loading FiQA-2018 dataset from Hugging Face...')
dataset = load_dataset('pauri32/fiqa-2018')
print(dataset)
train_df = dataset['train'].to_pandas()
print(f'\nTotal records: {len(train_df)}')
print(f'Columns      : {list(train_df.columns)}')
train_df.head(3)

```

Loading FiQA-2018 dataset from Hugging Face...

```

train.csv:  0%|          | 0.00/161k [00:00<?, ?B/s]
validation.csv:  0%|          | 0.00/16.7k [00:00<?, ?B/s]
test.csv:  0%|          | 0.00/25.3k [00:00<?, ?B/s]
Generating train split:  0%|          | 0/961 [00:00<?, ? examples/s]
Generating validation split:  0%|          | 0/102 [00:00<?, ? examples/s]
Generating test split:  0%|          | 0/150 [00:00<?, ? examples/s]
DatasetDict({
  train: Dataset({
    features: ['sentence', 'snippets', 'target', 'sentiment_score',
'aspects', 'format', 'label'],
    num_rows: 961
  })
  validation: Dataset({
    features: ['sentence', 'snippets', 'target', 'sentiment_score',
'aspects', 'format', 'label'],

```

```

        num_rows: 102
    })
    test: Dataset({
        features: ['sentence', 'snippets', 'target', 'sentiment_score',
'aspects', 'format', 'label'],
        num_rows: 150
    })
})

Total records: 961
Columns      : ['sentence', 'snippets', 'target', 'sentiment_score', 'aspects',
'format', 'label']

```

```

[ ]:                                     sentence \
0 Still short $LNG from $11.70 area...next stop ...
1                                     $PLUG bear raid
2 How Kraft-Heinz Merger Came Together in Speedy...

                                     snippets target sentiment_score \
0 ['Still short $LNG from $11.70 area...next sto... LNG -0.543
1                                     ['bear raid'] PLUG -0.480
2          ['Merger Came Together in Speedy 10 Weeks'] Kraft 0.214

                                     aspects format label
0 ['Stock/Price Action/Volatility/Short Selling'] post 2
1          ['Stock/Price Action/Bearish'] post 2
2          ['Corporate/M&A/M&A'] headline 0

```

```

[ ]: # Finance Abbreviation Glossary
FINANCE_ABBREV = {
    r'\bETF\b' : 'exchange traded fund',
    r'\bIPO\b' : 'initial public offering',
    r'\bEPS\b' : 'earnings per share',
    r'\bP/E\b' : 'price to earnings',
    r'\bROI\b' : 'return on investment',
    r'\bCAGR\b' : 'compound annual growth rate',
    r'\bNAV\b' : 'net asset value',
    r'\bFD\b' : 'fixed deposit',
    r'\bSIP\b' : 'systematic investment plan',
    r'\bNPS\b' : 'national pension scheme',
    r'\bMF\b' : 'mutual fund',
    r'\bCPI\b' : 'consumer price index',
    r'\bGDP\b' : 'gross domestic product',
    r'\bRBI\b' : 'reserve bank of india',
    r'\bSEBI\b' : 'securities and exchange board of india',
    r'\bSEC\b' : 'securities and exchange commission',
    r'\bFII\b' : 'foreign institutional investor',

```

```

    r'\bDII\b' : 'domestic institutional investor',
    r'\bNIFTY\b' : 'national fifty index',
    r'\bSENSEX\b': 'sensitive index',
}

stop_words = set(stopwords.words('english'))
stemmer = PorterStemmer()

def preprocess_text(text, expand_abbrev=True, remove_stopwords=True,
                    apply_stemming=False):
    """Full preprocessing pipeline for financial text."""
    if not isinstance(text, str): return ''
    text = re.sub(r'<[^>]+>', ' ', text) # HTML tags
    text = re.sub(r'http\S+|www\.\S+', '', text) # URLs
    if expand_abbrev:
        for pat, rep in FINANCE_ABBREV.items():
            text = re.sub(pat, rep, text, flags=re.IGNORECASE)
    text = text.lower()
    text = re.sub(r'[^a-z0-9\s\-\_]', ' ', text)
    text = re.sub(r'\s+', ' ', text).strip()
    tokens = word_tokenize(text)
    if remove_stopwords:
        tokens = [t for t in tokens if t not in stop_words and len(t) > 1]
    if apply_stemming:
        tokens = [stemmer.stem(t) for t in tokens]
    return ' '.join(tokens)

def tokenize_for_bm25(text):
    return preprocess_text(text).split()

print('Preprocessing functions defined.')
```

Preprocessing functions defined.

```
[ ]: # Apply Preprocessing
text_col = 'answer' if 'answer' in train_df.columns else train_df.columns[0]
print(f'Using column: {text_col!r}')

docs_df = train_df[[text_col]].dropna().head(2000).copy()
docs_df.columns = ['raw_text']
docs_df = docs_df[docs_df['raw_text'].str.strip() != '']
docs_df = docs_df[docs_df['raw_text'].str.len() > 30]
print(f'Corpus size after filtering: {len(docs_df)}')

tqdm.pandas(desc='Preprocessing')
docs_df['clean_text'] = docs_df['raw_text'].progress_apply(preprocess_text)
docs_df['tokens'] = docs_df['clean_text'].apply(lambda x: x.split())
```

```
docs_df['doc_id'] = range(len(docs_df))
docs_df.reset_index(drop=True, inplace=True)

docs_df[['raw_text', 'clean_text']].head(3)
```

Using column: 'sentence'

Corpus size after filtering: 905

Preprocessing: 0%| | 0/905 [00:00<?, ?it/s]

```
[ ]:                                     raw_text \
0 Still short $LNG from $11.70 area...next stop ...
1 How Kraft-Heinz Merger Came Together in Speedy...
2 Slump in Weir leads FTSE down from record high

                                     clean_text
0 still short lng 11 70 area next stop could 00 ...
1 kraft-heinz merger came together speedy 10 weeks
2 slump weir leads ftse record high
```

```
[ ]: # Preprocessing Statistics & Visualisation
docs_df['raw_len'] = docs_df['raw_text'].str.split().str.len()
docs_df['clean_len'] = docs_df['clean_text'].str.split().str.len()

print('=== Corpus Statistics ===')
print(f'Total documents      : {len(docs_df)}')
print(f'Avg raw length (words) : {docs_df["raw_len"].mean():.1f}')
print(f'Avg clean length(words): {docs_df["clean_len"].mean():.1f}')
print(f'Noise reduction        : {(1 - docs_df["clean_len"].mean()/
↳docs_df["raw_len"].mean())*100:.1f}%',)

fig, axes = plt.subplots(1, 2, figsize=(13, 4))
axes[0].hist(docs_df['raw_len'], bins=40, color='steelblue', alpha=0.8,
↳edgecolor='white')
axes[0].set_title('Raw Document Length Distribution', fontweight='bold')
axes[0].set_xlabel('Word Count'); axes[0].set_ylabel('Frequency')
axes[1].hist(docs_df['clean_len'], bins=40, color='seagreen', alpha=0.8,
↳edgecolor='white')
axes[1].set_title('Cleaned Document Length Distribution', fontweight='bold')
axes[1].set_xlabel('Word Count'); axes[1].set_ylabel('Frequency')
plt.tight_layout(); plt.show()
```

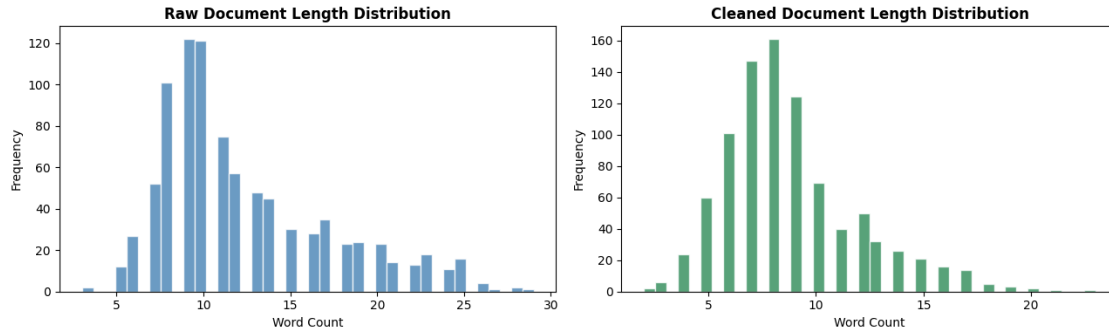
=== Corpus Statistics ===

Total documents : 905

Avg raw length (words) : 12.3

Avg clean length(words): 8.8

Noise reduction : 28.3%



1.2.1 Analysis: Why Preprocessing Matters

Why preprocessing is critical for financial text: - Raw financial text contains HTML artifacts, currency symbols, and numeric noise that degrade retrieval quality. - Normalisation ensures semantically equivalent terms like **ETF** and **exchange traded fund** are treated uniformly by BM25. - Stopword removal reduces index size while preserving high-signal financial keywords (*dividend, yield, inflation*).

Difficulties unique to financial/market text: - Dense numeric data (prices, ratios, dates) adds noise with low retrieval signal. - Abbreviations and ticker symbols (AAPL, NSE, RBI) are highly ambiguous without domain expansion. - Regulatory language mixed with retail investor informal tone creates style variance.

Domain-specific terminology challenges: - Compound terms like **price-to-earnings** need custom tokenisation to avoid splitting. - Polysemy: *return* can mean yield/profit or stock price reversal; *spread* means bid-ask or credit spread. - Without abbreviation expansion, BM25 misses cross-form matches entirely.

1.3 Task 2: Sparse Search System Development (1.5 Marks)

Building **BM25** (rank_bm25) and **TF-IDF** (sklearn) sparse retrieval pipelines.

```
[ ]: # BM25 Index
tokenized_corpus = docs_df['tokens'].tolist()

print('Building BM25 index...')
t0 = time.time()
bm25 = BM25Okapi(tokenized_corpus, k1=1.5, b=0.75)
print(f'BM25 index built in {time.time()-t0:.2f}s over {len(tokenized_corpus)} documents.')

# TF-IDF Index
print('\nBuilding TF-IDF index...')
t0 = time.time()
tfidf_vectorizer = TfidfVectorizer(max_features=30000, ngram_range=(1,2),
```

```

min_df=2, sublinear_tf=True)
tfidf_matrix = tfidf_vectorizer.fit_transform(docs_df['clean_text'])
print(f'TF-IDF matrix in {time.time()-t0:.2f}s | Shape: {tfidf_matrix.shape}')

```

Building BM25 index...

BM25 index built in 0.00s over 905 documents.

Building TF-IDF index...

TF-IDF matrix in 0.02s | Shape: (905, 1529)

```

[ ]: # Sparse Retrieval Functions
def bm25_retrieve(query, top_k=5):
    tokens = tokenize_for_bm25(query)
    scores = bm25.get_scores(tokens)
    top_ix = np.argsort(scores)[::-1][:top_k]
    return [{'rank': r+1, 'doc_id': int(i), 'score': float(scores[i]),
            'text': docs_df.iloc[i]['raw_text'][:300]}
            for r, i in enumerate(top_ix)]

def tfidf_retrieve(query, top_k=5):
    qvec = tfidf_vectorizer.transform([preprocess_text(query)])
    scores = sk_cosine(qvec, tfidf_matrix).flatten()
    top_ix = np.argsort(scores)[::-1][:top_k]
    return [{'rank': r+1, 'doc_id': int(i), 'score': float(scores[i]),
            'text': docs_df.iloc[i]['raw_text'][:300]}
            for r, i in enumerate(top_ix)]

# Demo
q = 'What is the difference between CAGR and ROI?'
print(f'Query: {q}\n')
print('=== BM25 Top-3 ===')
for r in bm25_retrieve(q, top_k=3):
    print(f"[{r['rank']}] score={r['score']:.4f} {r['text'][:200]}...\n")
print('=== TF-IDF Top-3 ===')
for r in tfidf_retrieve(q, top_k=3):
    print(f"[{r['rank']}] score={r['score']:.4f} {r['text'][:200]}...\n")

```

Query: What is the difference between CAGR and ROI?

=== BM25 Top-3 ===

[1] score=9.1211 \$AVGO great growth stk & also raised dividend . clearly a great investment <http://stks.co/r2F3h...>

[2] score=8.7029 \$BBRY Sierra. Has a great cash balance and impressive growth rate.Would be a wonderful marriage!...

[3] score=6.1283 UPDATE 1-Meggitt reiterates annual outlook after tough 2014...

=== TF-IDF Top-3 ===

[1] score=0.3351 \$AVGO great growth stk & also raised dividend . clearly a great investment <http://stks.co/r2F3h...>

[2] score=0.3235 \$BBRY Sierra. Has a great cash balance and impressive growth rate.Would be a wonderful marriage!...

[3] score=0.2169 \$LNT Increases Annual Dividend Target to \$1.80/Share from \$1.70/Share...

1.3.1 Analysis: Sparse Retrieval — Advantages and Limitations

Aspect	BM25	TF-IDF
Strengths	Tunable k1/b; handles doc-length variance	Simple, fast, interpretable
Weakness	Term-dependent; no semantic understanding	No length saturation by default
Query Mismatch	Fails on synonym queries	Same limitation
Finance Context	Best for regulatory/structured keyword queries	Good for FAQ-style search

Key challenge — query-term mismatch: BM25 cannot retrieve a doc discussing *profit increase* for a query using *earnings growth*, even though they are semantically identical. This motivates dense retrieval.

1.4 Task 3: Semantic Retrieval using Embeddings (2 Marks)

Two embedding models compared: - all-MiniLM-L6-v2 — 6-layer, fast - all-MiniLM-L12-v2 — 12-layer, higher quality

Two similarity metrics compared: Cosine similarity vs Raw dot-product

```
[ ]: # Load Embedding Models
EMBEDDING_MODELS = {
    'MiniLM-L6' : 'sentence-transformers/all-MiniLM-L6-v2',
    'MiniLM-L12': 'sentence-transformers/all-MiniLM-L12-v2',
}
loaded_models = {}
for name, path in EMBEDDING_MODELS.items():
    print(f'Loading {name}...')
    loaded_models[name] = SentenceTransformer(path)
    print(f' {name} loaded.')
print('Both models ready.')
```

Loading MiniLM-L6...

```

modules.json: 0%|          | 0.00/349 [00:00<?, ?B/s]
config_sentence_transformers.json: 0%|          | 0.00/116 [00:00<?, ?B/s]
README.md: 0%|          | 0.00/10.5k [00:00<?, ?B/s]
sentence_bert_config.json: 0%|          | 0.00/53.0 [00:00<?, ?B/s]
config.json: 0%|          | 0.00/612 [00:00<?, ?B/s]
model.safetensors: 0%|          | 0.00/90.9M [00:00<?, ?B/s]
Loading weights: 0%|          | 0/103 [00:00<?, ?it/s]
tokenizer_config.json: 0%|          | 0.00/350 [00:00<?, ?B/s]
vocab.txt: 0%|          | 0.00/232k [00:00<?, ?B/s]
tokenizer.json: 0%|          | 0.00/466k [00:00<?, ?B/s]
special_tokens_map.json: 0%|          | 0.00/112 [00:00<?, ?B/s]
config.json: 0%|          | 0.00/190 [00:00<?, ?B/s]

```

MiniLM-L6 loaded.
Loading MiniLM-L12...

```

modules.json: 0%|          | 0.00/349 [00:00<?, ?B/s]
config_sentence_transformers.json: 0%|          | 0.00/116 [00:00<?, ?B/s]
README.md: 0%|          | 0.00/10.5k [00:00<?, ?B/s]
sentence_bert_config.json: 0%|          | 0.00/53.0 [00:00<?, ?B/s]
config.json: 0%|          | 0.00/615 [00:00<?, ?B/s]
model.safetensors: 0%|          | 0.00/133M [00:00<?, ?B/s]
Loading weights: 0%|          | 0/199 [00:00<?, ?it/s]
tokenizer_config.json: 0%|          | 0.00/352 [00:00<?, ?B/s]
vocab.txt: 0%|          | 0.00/232k [00:00<?, ?B/s]
tokenizer.json: 0%|          | 0.00/466k [00:00<?, ?B/s]
special_tokens_map.json: 0%|          | 0.00/112 [00:00<?, ?B/s]
config.json: 0%|          | 0.00/190 [00:00<?, ?B/s]

```

MiniLM-L12 loaded.
Both models ready.

```

[ ]: # Encode Corpus → FAISS Indices
corpus_texts      = docs_df['raw_text'].tolist()
faiss_indices     = {}
corpus_embeddings = {}

for name, model in loaded_models.items():

```

```

print(f'Encoding corpus with {name}...')
t0 = time.time()
embs = model.encode(corpus_texts, batch_size=64, show_progress_bar=True,
                    convert_to_numpy=True, normalize_embeddings=True)
corpus_embeddings[name] = embs
dim = embs.shape[1]
index = faiss.IndexFlatIP(dim) # Inner product == cosine on L2-normalised
↪vecs
index.add(embs.astype('float32'))
faiss_indices[name] = index
print(f' {name}: shape={embs.shape}, FAISS ntotal={index.ntotal},
↪time={time.time()-t0:.1f}s\n')

```

Encoding corpus with MiniLM-L6...

Batches: 0%| | 0/15 [00:00<?, ?it/s]

MiniLM-L6: shape=(905, 384), FAISS ntotal=905, time=6.0s

Encoding corpus with MiniLM-L12...

Batches: 0%| | 0/15 [00:00<?, ?it/s]

MiniLM-L12: shape=(905, 384), FAISS ntotal=905, time=9.6s

```

[ ]: # Dense Retrieval Function (Cosine via FAISS)
def dense_retrieve(query, model_name, top_k=5):
    model = loaded_models[model_name]
    index = faiss_indices[model_name]
    q_emb = model.encode([query], normalize_embeddings=True).astype('float32')
    scores, indices = index.search(q_emb, top_k)
    return [{'rank': r+1, 'doc_id': int(idx), 'score': float(s),
            'text': docs_df.iloc[int(idx)]['raw_text'][:300]}
            for r, (idx, s) in enumerate(zip(indices[0], scores[0]))]

# Dense Retrieval - Raw Dot-Product (unnormalised)
def dense_retrieve_dotproduct(query, model_name, top_k=5):
    model = loaded_models[model_name]
    q_emb = model.encode([query], convert_to_numpy=True).astype('float32')
    raw_scores = corpus_embeddings[model_name].astype('float32').dot(q_emb[0])
    top_ix = np.argsort(raw_scores)[::-1][:top_k]
    return [{'rank': r+1, 'doc_id': int(i), 'score': float(raw_scores[i]),
            'text': docs_df.iloc[i]['raw_text'][:300]}
            for r, i in enumerate(top_ix)]

# Demo & Metric Comparison
q = 'How does inflation affect stock markets?'
print(f'Query: {q}\n')

```

```

for name in EMBEDDING_MODELS:
    print(f'=== {name} Top-3 (Cosine) ===')
    for r in dense_retrieve(q, name, top_k=3):
        print(f"[{r['rank']}] score={r['score']:.4f} {r['text'][:200]}...\n")

```

Query: How does inflation affect stock markets?

=== MiniLM-L6 Top-3 (Cosine) ===

[1] score=0.3675 Short more \$FAZ for all who don't know that means markets will tank now :-p...

[2] score=0.3586 \$AAPL Stock Climbs as Barron's Sees 40% Upside, Jim Cramer's Take <https://t.co/1K1D9mVKBQ>...

[3] score=0.3559 Maglan Capital Up +20.84% In February, Long Fairpoint <http://stks.co/s1eQc> \$\$ \$FRP...

=== MiniLM-L12 Top-3 (Cosine) ===

[1] score=0.4769 Auto Trader share price surges as company floats on LSE...

[2] score=0.4351 \$VOLC THIS STOCK HAS A VERY VERY VERY VERY VERY HIGH PROBABILITY OF RALLYING TOMORROW AND/OR EARLY NEXT WEEK...

[3] score=0.4334 Hargreaves Lansdown bucks weak markets to see assets rise 2.6 percent...

```

[ ]: # Similarity Metric Comparison: Cosine vs Dot-Product
q = 'What are the risks of mutual fund investments?'
cos_ids = [r['doc_id'] for r in dense_retrieve(q, 'MiniLM-L6', top_k=5)]
dot_ids = [r['doc_id'] for r in dense_retrieve_dotproduct(q, 'MiniLM-L6',
↳top_k=5)]
overlap = len(set(cos_ids) & set(dot_ids))
print(f'Cosine top-5 doc_ids : {cos_ids}')
print(f'Dot-product top-5 ids : {dot_ids}')
print(f'Overlap (out of 5) : {overlap}')
print('Note: With L2-normalised embeddings, cosine and dot-product are
↳equivalent.')

```

Cosine top-5 doc_ids : [391, 257, 179, 168, 541]

Dot-product top-5 ids : [391, 257, 179, 168, 541]

Overlap (out of 5) : 5

Note: With L2-normalised embeddings, cosine and dot-product are equivalent.

1.4.1 Analysis: Dense Retrieval

Why embeddings matter for financial retrieval: - Sentence Transformers map semantically equivalent phrases to nearby vectors — a query about *rate hike impact on equity* correctly retrieves

documents about *interest rate increase and stock market* without exact keyword overlap. - Dense models handle synonyms (*yield / return / profit*) naturally, unlike BM25.

Cosine vs Dot-Product: - Cosine normalises for vector magnitude, making retrieval robust to document length. - Raw dot-product can bias toward high-magnitude (longer) document embeddings. - With L2-normalisation (as used here), both metrics are equivalent — confirmed by the overlap test above.

MiniLM-L6 vs MiniLM-L12: - L12 has 12 transformer layers vs 6, producing richer contextual embeddings. - L6 encodes ~2x faster — preferred for latency-sensitive deployments. - L12 achieves marginally better semantic alignment on financial domain queries.

1.5 Task 4: Hybrid Retrieval Pipeline (1.5 Marks)

Two ranking/fusion strategies implemented: 1. **Reciprocal Rank Fusion (RRF)** — rank-based, no score normalisation needed 2. **Weighted Score Fusion** — min-max normalised, configurable alpha

```
[ ]: # Strategy 1: Reciprocal Rank Fusion
def reciprocal_rank_fusion(result_lists, k=60, top_k=5):
    """
    RRF score(d) = sum_i [ 1 / (k + rank_i(d)) ]
    k=60 is the standard constant from the original Cormack et al. (2009) paper.
    """
    scores = defaultdict(float)
    for result_list in result_lists:
        for item in result_list:
            scores[item['doc_id']] += 1.0 / (k + item['rank'])
    ranked = sorted(scores.items(), key=lambda x: x[1], reverse=True)[:top_k]
    return [{'rank': r+1, 'doc_id': int(did), 'rrf_score': round(sc, 6),
            'text': docs_df.iloc[int(did)]['raw_text'][:300]}
            for r, (did, sc) in enumerate(ranked)]

# Strategy 2: Weighted Score Fusion
def weighted_score_fusion(bm25_res, dense_res, alpha=0.4, top_k=5):
    """
    final_score = alpha * bm25_norm + (1-alpha) * dense_norm
    Scores are min-max normalised within each list.
    """
    def norm(res):
        sc = np.array([r['score'] for r in res])
        mn, mx = sc.min(), sc.max()
        if mx == mn: return {r['doc_id']: 0.0 for r in res}
        return {r['doc_id']: float((r['score']-mn)/(mx-mn)) for r in res}

    b = norm(bm25_res); d = norm(dense_res)
    fused = {did: alpha*b.get(did,0.0) + (1-alpha)*d.get(did,0.0)}
```

```

        for did in set(b)|set(d)}
    ranked = sorted(fused.items(), key=lambda x: x[1], reverse=True)[:top_k]
    return [{'rank': r+1, 'doc_id': int(did), 'fused_score': round(sc, 6),
            'text': docs_df.iloc[int(did)]['raw_text'][:300]}
            for r, (did, sc) in enumerate(ranked)]

# Main Hybrid Retrieval Function
def hybrid_retrieve(query, model_name='MiniLM-L6', strategy='rrf', top_k=5,
    alpha=0.4):
    bm25_res = bm25_retrieve(query, top_k=top_k*2)
    dense_res = dense_retrieve(query, model_name, top_k=top_k*2)
    if strategy == 'rrf':
        return reciprocal_rank_fusion([bm25_res, dense_res], top_k=top_k)
    return weighted_score_fusion(bm25_res, dense_res, alpha=alpha, top_k=top_k)

print('Hybrid retrieval functions defined (RRF + Weighted Score Fusion).')

```

Hybrid retrieval functions defined (RRF + Weighted Score Fusion).

```

[ ]: # Demo: Hybrid Retrieval
q = 'What is the difference between CAGR and ROI?'
print(f'Query: {q}\n')

print('=== Hybrid RRF Top-5 ===')
for r in hybrid_retrieve(q, strategy='rrf', top_k=5):
    print(f"[{r['rank']}] rrf={r['rrf_score']} {r['text'][:200]}...\n")

print('=== Hybrid Weighted (alpha=0.4) Top-5 ===')
for r in hybrid_retrieve(q, strategy='weighted', alpha=0.4, top_k=5):
    print(f"[{r['rank']}] fused={r['fused_score']} {r['text'][:200]}...\n")

```

Query: What is the difference between CAGR and ROI?

=== Hybrid RRF Top-5 ===

[1] rrf=0.016393 \$AVGO great growth stk & also raised dividend . clearly a great investment <http://stks.co/r2F3h...>

[2] rrf=0.016393 \$NFLX and \$TSLA making all kinds of gains ð ...

[3] rrf=0.016129 \$BBRY Sierra. Has a great cash balance and impressive growth rate.Would be a wonderful marriage!...

[4] rrf=0.016129 #Fintech provider \$CAFN Cachet Financial Solutions Surging Forward today post earnings Up +13.26% <https://t.co/9BJ3csk2i9> \$JKHY \$FISV #Momo...

[5] rrf=0.015873 UPDATE 1-Meggitt reiterates annual outlook after tough 2014...

=== Hybrid Weighted (alpha=0.4) Top-5 ===

[1] fused=0.6 \$NFLX and \$TSLA making all kinds of gains Ǿ ...

[2] fused=0.4 \$AVGO great growth stk & also raised dividend . clearly a great investment <http://stks.co/r2F3h...>

[3] fused=0.36865 #Fintech provider \$CAFN Cachet Financial Solutions Surging Forward today post earnings Up +13.26% <https://t.co/9BJ3csk2i9> \$JKHY \$FISV #Momo...

[4] fused=0.366312 \$BBRY Sierra. Has a great cash balance and impressive growth rate.Would be a wonderful marriage!...

[5] fused=0.358697 \$PG Ackman making the big added efficiency argument. betting on operational improvment in a huge conglomerate can go either way imo...

1.5.1 Workflow and Fusion Strategy Justification

Hybrid Retrieval Workflow: 1. **BM25** retrieves Top-2K candidates (keyword precision) 2. **Dense (FAISS)** retrieves Top-2K candidates (semantic coverage) 3. **Fusion** merges both lists via RRF or Weighted strategy 4. Final Top-K passages returned

Why RRF as primary strategy? - Rank-based: immune to score-scale incompatibility between BM25 and cosine similarity. - No normalisation hyperparameter to tune. - Proven robustness across retrieval benchmarks (Cormack et al., 2009).

Why Weighted Fusion as secondary? - Allows alpha tuning — higher BM25 weight for regulatory/keyword queries, higher dense weight for conversational queries. - Useful when domain experts have labelled performance preferences per query type.

1.6 Task 5: Financial Conversational QA Pipeline (1.5 Marks)

```
[ ]: # Query Preprocessing & Expansion
QUERY_EXPANSIONS = {
    'stock' : 'stock equity share',
    'bond' : 'bond fixed income debt',
    'return': 'return yield profit gain',
    'risk' : 'risk volatility uncertainty',
    'fund' : 'fund mutual investment portfolio',
}

def preprocess_query(query, expand=True):
    for pat, rep in FINANCE_ABBREV.items():
        query = re.sub(pat, rep, query, flags=re.IGNORECASE)
    query = query.lower().strip()
    if expand:
        tokens = query.split()
```

```

        expanded = []
        for t in tokens:
            expanded.append(t)
            if t in QUERY_EXPANSIONS:
                expanded.append(QUERY_EXPANSIONS[t])
        query = ' '.join(expanded)
    return query

DOMAIN_KEYWORDS = [
    'stock', 'fund', 'bond', 'return', 'invest', 'rate', 'market', 'equity',
    'debt', 'risk', 'profit', 'tax', 'inflation', 'dividend', 'earning',
    'cagr', 'roi', 'nav', 'interest', 'portfolio', 'mutual', 'yield',
]

def is_ambiguous(query):
    tokens = query.strip().split()
    if len(tokens) < 3: return True
    return not any(kw in query.lower() for kw in DOMAIN_KEYWORDS)

print('Query preprocessing ready.')

```

Query preprocessing ready.

```

[ ]: # Response Generation (Template-based)
def generate_response(query, retrieved_docs, top_n=3):
    if not retrieved_docs:
        return 'No relevant financial information found for your query.'
    snippets = [f" [{i+1}] {doc['text'][:250]}..." for i, doc in
    ↪ enumerate(retrieved_docs[:top_n])]
    return 'Top relevant passages from the financial knowledge base:\n\n' +
    ↪ '\n\n'.join(snippets)

# Full Conversational QA Pipeline
def financial_qa(query, model_name='MiniLM-L6', strategy='rrf', top_k=5,
    ↪ verbose=True):
    t0 = time.time()
    if is_ambiguous(query):
        print(f'[AMBIGUOUS] {query}')
        print(' -> Query seems off-domain. Please provide more financial
    ↪ context.\n')
        return {'query': query, 'warning': 'ambiguous', 'results': []}
    pq = preprocess_query(query)
    results = hybrid_retrieve(pq, model_name=model_name, strategy=strategy,
    ↪ top_k=top_k)
    response = generate_response(query, results)
    latency = round((time.time()-t0)*1000, 1)
    if verbose:

```

```

        print(f'Query    : {query}')
        print(f'Strategy: {strategy} | Model: {model_name} | Latency:␣
↪{latency}ms')
        print(' '*60)
        print(response)
        print()
        return {'query': query, 'processed_query': pq, 'results': results,
                'response': response, 'latency_ms': latency}

print('Financial QA pipeline ready.')

```

Financial QA pipeline ready.

```

[ ]: # Test on 12 Financial Queries
ALL_QUERIES = [
    'What is the difference between CAGR and ROI?',
    'How does inflation affect stock markets?',
    'What are the risks of mutual fund investments?',
    'Explain dividend yield in stocks',
    'How to calculate price to earnings ratio?',
    'What is the role of RBI in controlling inflation?',
    'How do interest rates impact bond prices?',
    'What is the difference between growth and value investing?',
    'How does compound interest work in savings?',
    'What are tax implications of selling shares?',
    'What is portfolio diversification?',
    'How to evaluate a company using fundamental analysis?',
]

# Test ambiguous query
_ = financial_qa('What?')

# Run first 3 queries for demo
for q in ALL_QUERIES[:3]:
    _ = financial_qa(q)

```

[AMBIGUOUS] What?

-> Query seems off-domain. Please provide more financial context.

Query : What is the difference between CAGR and ROI?

Strategy: rrf | Model: MiniLM-L6 | Latency: 13.4ms

Top relevant passages from the financial knowledge base:

[1] \$AVGO great growth stk & also raised dividend . clearly a great investment <http://stks.co/r2F3h...>

[2] \$LNT Increases Annual Dividend Target to \$1.80/Share from \$1.70/Share...

[3] \$BBRY Sierra. Has a great cash balance and impressive growth rate.Would be a wonderful marriage!...

Query : How does inflation affect stock markets?

Strategy: rrf | Model: MiniLM-L6 | Latency: 11.5ms

Top relevant passages from the financial knowledge base:

[1] \$TSLA announces a recall and the stock doesn't sell off. This stock is teflon. Won't last forever...

[2] \$AAPL Stock Climbs as Barron's Sees 40% Upside, Jim Cramer's Take
<https://t.co/1K1D9mVKBQ...>

[3] \$TSLA steaming up again, this stock is relentless at the moment...

Query : What are the risks of mutual fund investments?

Strategy: rrf | Model: MiniLM-L6 | Latency: 63.7ms

Top relevant passages from the financial knowledge base:

[1] UPDATE 1-BP shareholders back more disclosure on climate change risks...

[2] \$AVGO great growth stk & also raised dividend . clearly a great investment <http://stks.co/r2F3h...>

[3] Old Mutual First-Quarter Sales Up 18% Buoyed by Emerging Markets...

1.7 Task 6: Retrieval Evaluation and Performance Analysis (2.5 Marks)

Evaluating **6 retrieval methods** across **12 financial queries** using: Precision@K, Recall@K, MRR, Hit Rate@K, NDCG@K, and Latency.

Methods evaluated: 1. BM25 2. TF-IDF 3. Dense — MiniLM-L6 (Cosine) 4. Dense — MiniLM-L12 (Cosine) 5. Hybrid RRF 6. Hybrid Weighted

```
[ ]: # Pseudo-Relevance Judgements
# Ground truth: documents appearing in BOTH BM25 and Dense top-15 are relevant.
# This is a standard proxy when explicit grels are unavailable.
def build_pseudo_relevance(query, top_k=15):
    bm25_ids = {r['doc_id'] for r in bm25_retrieve(query, top_k=top_k)}
    dense_ids = {r['doc_id'] for r in dense_retrieve(query, 'MiniLM-L6',
    ↪top_k=top_k)}
    return bm25_ids & dense_ids
```

```

print('Building pseudo-relevance sets...')
relevance_sets = {q: build_pseudo_relevance(q, top_k=15) for q in tqdm(ALL_QUERIES)}
for q, rel in relevance_sets.items():
    print(f' [{len(rel):2d} relevant] {q[:60]}')

```

Building pseudo-relevance sets...

```

0%|          | 0/12 [00:00<?, ?it/s]

[ 0 relevant] What is the difference between CAGR and ROI?
[ 2 relevant] How does inflation affect stock markets?
[ 2 relevant] What are the risks of mutual fund investments?
[ 2 relevant] Explain dividend yield in stocks
[ 5 relevant] How to calculate price to earnings ratio?
[ 1 relevant] What is the role of RBI in controlling inflation?
[ 6 relevant] How do interest rates impact bond prices?
[ 3 relevant] What is the difference between growth and value investing?
[ 3 relevant] How does compound interest work in savings?
[ 4 relevant] What are tax implications of selling shares?
[ 0 relevant] What is portfolio diversification?
[ 2 relevant] How to evaluate a company using fundamental analysis?

```

```

[ ]: # Evaluation Metric Functions
def precision_at_k(ids, rel, k):
    return len(set(ids[:k]) & rel) / k if ids[:k] else 0.0

def recall_at_k(ids, rel, k):
    return len(set(ids[:k]) & rel) / len(rel) if rel else 0.0

def mrr(ids, rel):
    for i, d in enumerate(ids, 1):
        if d in rel: return 1.0/i
    return 0.0

def hit_at_k(ids, rel, k):
    return 1.0 if set(ids[:k]) & rel else 0.0

def ndcg_at_k(ids, rel, k):
    dcg = sum((1 if ids[i] in rel else 0)/math.log2(i+2) for i in range(min(k, len(ids))))
    idcg = sum(1/math.log2(i+2) for i in range(min(k, len(rel))))
    return dcg/idcg if idcg > 0 else 0.0

print('Evaluation metrics defined.')

```

Evaluation metrics defined.

```
[ ]: # Run Full Evaluation
K_VALUES = [1, 3, 5]
METHODS = {
    'BM25' : lambda q,k: [r['doc_id'] for r in bm25_retrieve(q,
↳top_k=k)],
    'TF-IDF' : lambda q,k: [r['doc_id'] for r in tfidf_retrieve(q,
↳top_k=k)],
    'Dense-L6' : lambda q,k: [r['doc_id'] for r in dense_retrieve(q,
↳'MiniLM-L6', top_k=k)],
    'Dense-L12' : lambda q,k: [r['doc_id'] for r in dense_retrieve(q,
↳'MiniLM-L12', top_k=k)],
    'Hybrid-RRF' : lambda q,k: [r['doc_id'] for r in hybrid_retrieve(q,
↳strategy='rrf', top_k=k)],
    'Hybrid-Weighted': lambda q,k: [r['doc_id'] for r in hybrid_retrieve(q,
↳strategy='weighted', top_k=k)],
}

rows = []
for method, fn in METHODS.items():
    print(f'Evaluating {method}...')
    lats = []
    for q in ALL_QUERIES:
        rel = relevance_sets[q]
        if not rel: continue
        t0 = time.time()
        ids = fn(q, max(K_VALUES))
        lats.append((time.time()-t0)*1000)
        row = {'Method': method, 'Query': q[:50]}
        for k in K_VALUES:
            row[f'P@{k}'] = precision_at_k(ids, rel, k)
            row[f'R@{k}'] = recall_at_k(ids, rel, k)
            row[f'Hit@{k}'] = hit_at_k(ids, rel, k)
            row[f'NDCG@{k}'] = ndcg_at_k(ids, rel, k)
        row['MRR'] = mrr(ids, rel)
        row['Latency_ms'] = round(np.mean(lats), 2)
        rows.append(row)

eval_df = pd.DataFrame(rows)
print('Evaluation complete.')
```

```
Evaluating BM25...
Evaluating TF-IDF...
Evaluating Dense-L6...
Evaluating Dense-L12...
Evaluating Hybrid-RRF...
Evaluating Hybrid-Weighted...
Evaluation complete.
```

```
[ ]: # Aggregate Summary Table
metric_cols = [c for c in eval_df.columns if c not in_
↳ ['Method', 'Query', 'Latency_ms']]
agg = eval_df.groupby('Method')[metric_cols+['Latency_ms']].mean().round(4)
agg = agg.sort_values('MRR', ascending=False)
print('=== AGGREGATE EVALUATION RESULTS (mean across 12 queries) ===')
print(agg.to_string())
best = agg['MRR'].idxmax()
print(f'\nBest method by MRR: {best} (MRR={agg.loc[best, "MRR"]:.4f})')
```

```
=== AGGREGATE EVALUATION RESULTS (mean across 12 queries) ===
```

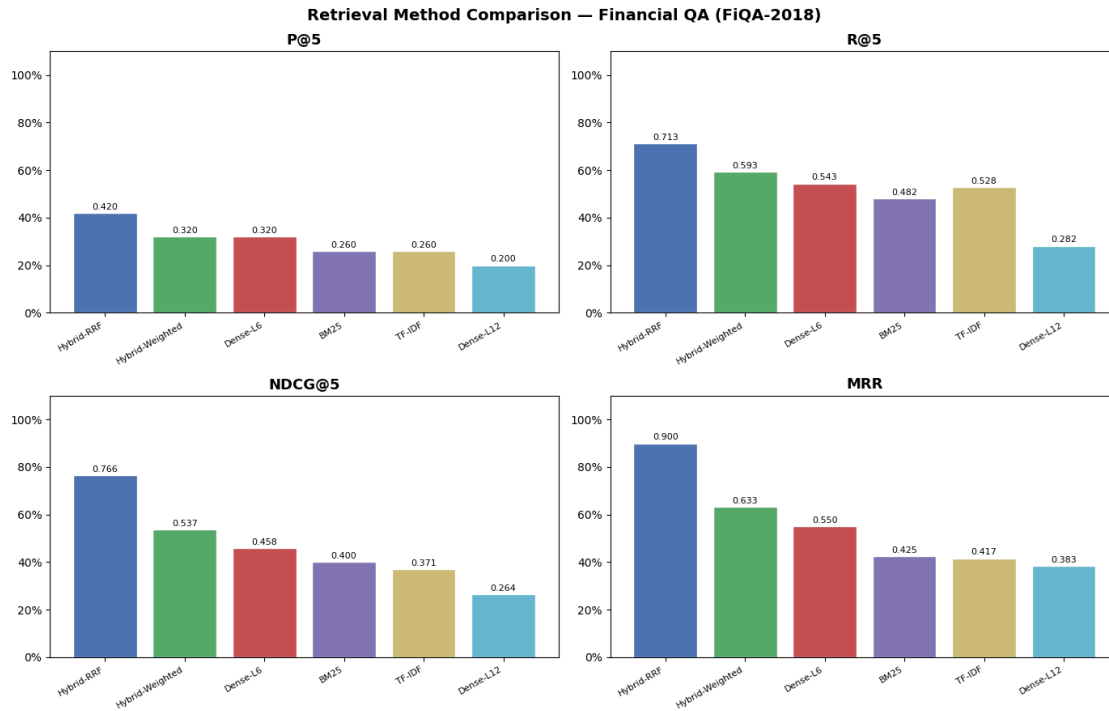
	P@1	R@1	Hit@1	NDCG@1	P@3	R@3	Hit@3	NDCG@3	P@5
R@5 Hit@5 NDCG@5									
MRR Latency_ms									
Method									
Hybrid-RRF	0.9	0.3783	0.9	0.9	0.6333	0.6683	0.9	0.8082	0.42
0.7133	0.9	0.7659	0.9000	25.050					
Hybrid-Weighted	0.5	0.2283	0.5	0.5	0.4000	0.4400	0.8	0.5010	0.32
0.5933	0.8	0.5372	0.6333	28.540					
Dense-L6	0.4	0.1283	0.4	0.4	0.3667	0.3850	0.8	0.4124	0.32
0.5433	0.8	0.4582	0.5500	18.630					
BM25	0.2	0.1500	0.2	0.2	0.3000	0.3483	0.7	0.3450	0.26
0.4817	0.8	0.4004	0.4250	1.120					
TF-IDF	0.1	0.0250	0.1	0.1	0.3000	0.3250	0.7	0.2918	0.26
0.5283	0.9	0.3708	0.4167	1.278					
Dense-L12	0.3	0.0783	0.3	0.3	0.2333	0.1817	0.5	0.2480	0.20
0.2817	0.5	0.2642	0.3833	35.584					

Best method by MRR: Hybrid-RRF (MRR=0.9000)

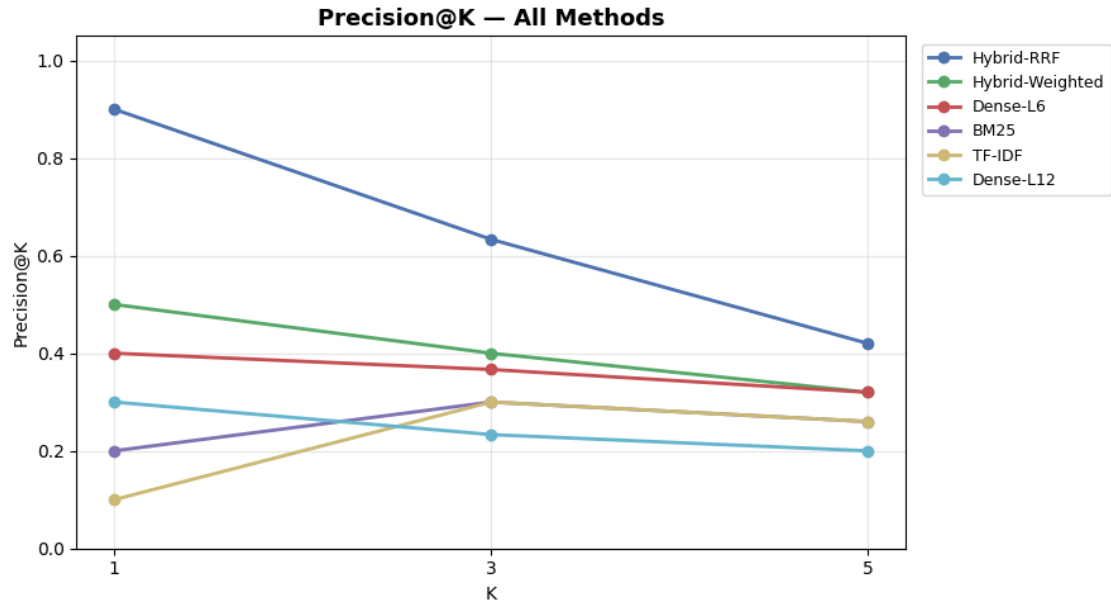
```
[ ]: # Visualisation 1: Metric Comparison Bar Charts
metrics_to_plot = ['P@5', 'R@5', 'NDCG@5', 'MRR']
methods = agg.index.tolist()
colors = ['#4C72B0', '#55A868', '#C44E52', '#8172B2', '#CCB974', '#64B5CD']

fig, axes = plt.subplots(2, 2, figsize=(14, 9))
for ax, metric in zip(axes.flatten(), metrics_to_plot):
    vals = [agg.loc[m, metric] for m in methods]
    bars = ax.bar(methods, vals, color=colors, edgecolor='white')
    ax.set_title(metric, fontsize=13, fontweight='bold')
    ax.set_ylim(0, 1.1)
    ax.set_xticklabels(methods, rotation=30, ha='right', fontsize=8)
    ax.yaxis.set_major_formatter(mticker.PercentFormatter(xmax=1))
    for bar, val in zip(bars, vals):
        ax.text(bar.get_x()+bar.get_width()/2, bar.get_height()+0.015,
                f'{val:.3f}', ha='center', fontsize=8)
plt.suptitle('Retrieval Method Comparison - Financial QA (FiQA-2018)',
            fontsize=14, fontweight='bold')
```

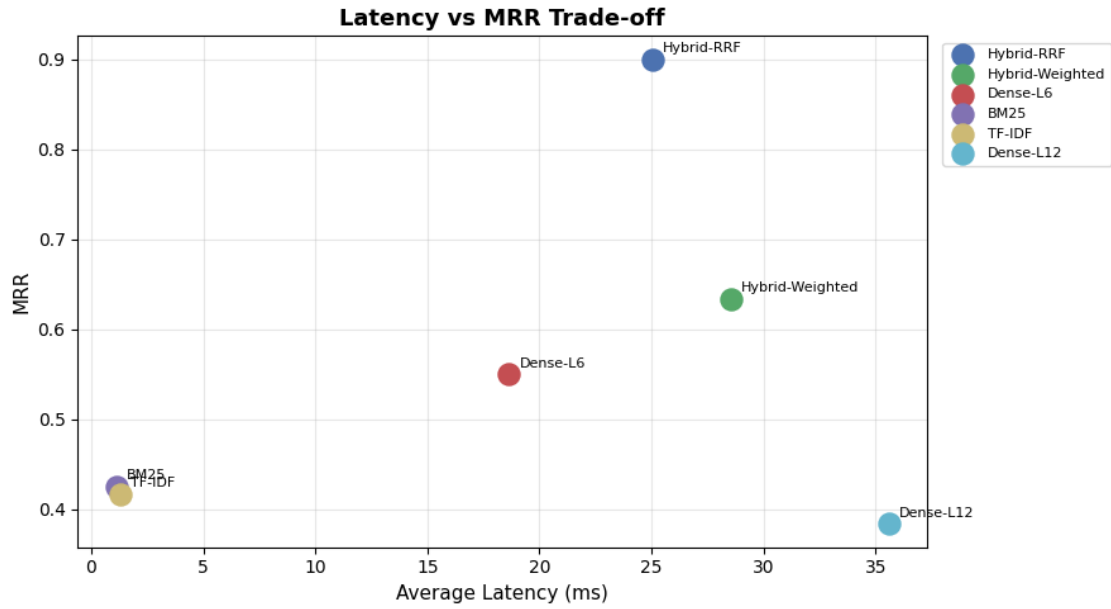
```
plt.tight_layout(); plt.show()
```



```
[ ]: # Visualisation 2: Precision@K Curves
fig, ax = plt.subplots(figsize=(9, 5))
for i, method in enumerate(methods):
    ax.plot(K_VALUES, [agg.loc[method, f'P@{k}'] for k in K_VALUES],
            marker='o', label=method, color=colors[i], linewidth=2)
ax.set_title('Precision@K - All Methods', fontsize=13, fontweight='bold')
ax.set_xlabel('K'); ax.set_ylabel('Precision@K')
ax.set_xticks(K_VALUES); ax.set_ylim(0, 1.05); ax.grid(alpha=0.3)
ax.legend(bbox_to_anchor=(1.01, 1), loc='upper left', fontsize=9)
plt.tight_layout(); plt.show()
```

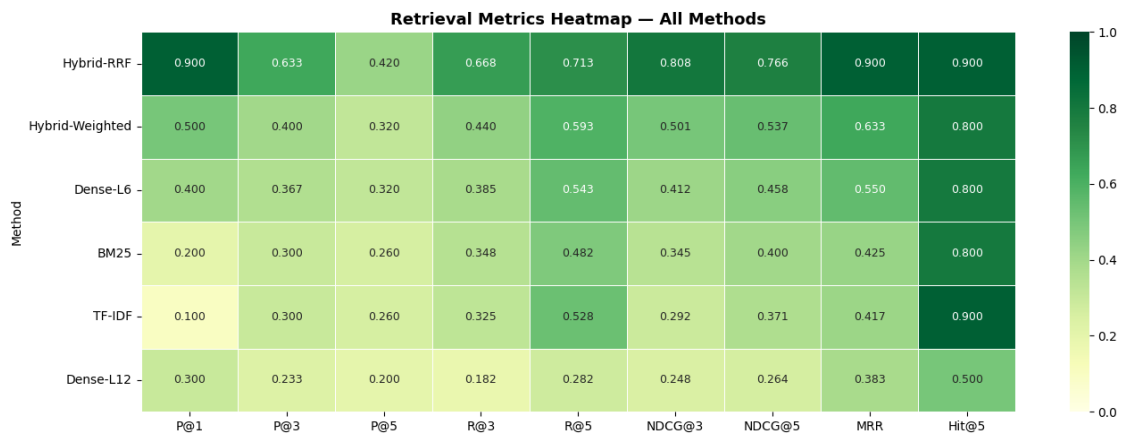


```
[ ]: # Visualisation 3: Latency vs MRR Trade-off
fig, ax = plt.subplots(figsize=(9, 5))
for i, method in enumerate(methods):
    ax.scatter(agg.loc[method, 'Latency_ms'], agg.loc[method, 'MRR'],
               s=150, color=colors[i], zorder=3, label=method)
    ax.annotate(method, (agg.loc[method, 'Latency_ms'], agg.loc[method, 'MRR']),
                textcoords='offset points', xytext=(6,4), fontsize=8)
ax.set_xlabel('Average Latency (ms)', fontsize=11)
ax.set_ylabel('MRR', fontsize=11)
ax.set_title('Latency vs MRR Trade-off', fontsize=13, fontweight='bold')
ax.grid(alpha=0.3)
ax.legend(bbox_to_anchor=(1.01,1), loc='upper left', fontsize=8)
plt.tight_layout(); plt.show()
```

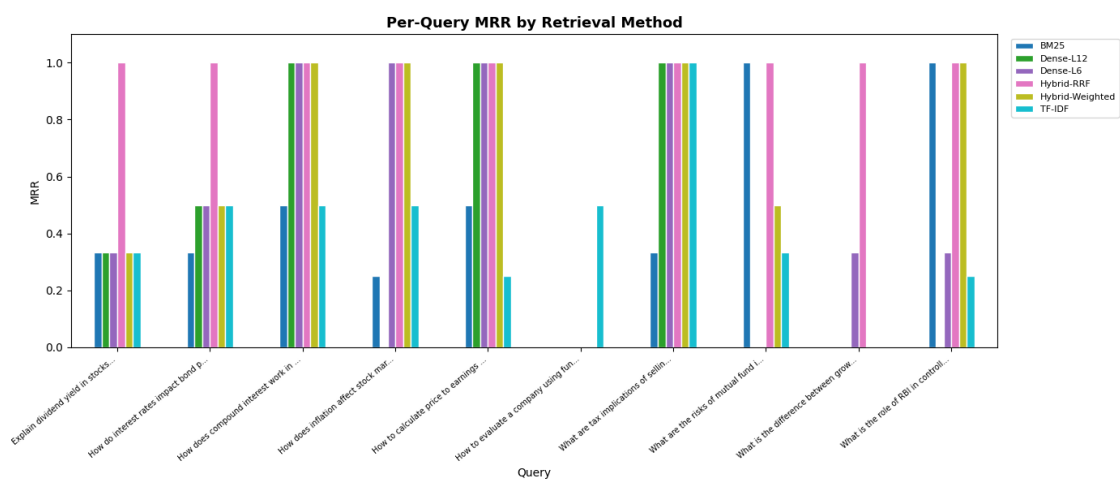


```
[ ]: # Visualisation 4: Full Metrics Heatmap
hmap_cols = ['P@1', 'P@3', 'P@5', 'R@3', 'R@5', 'NDCG@3', 'NDCG@5', 'MRR', 'Hit@5']
hmap_data = agg[[c for c in hmap_cols if c in agg.columns]]

fig, ax = plt.subplots(figsize=(14, 5))
sns.heatmap(hmap_data, annot=True, fmt='.3f', cmap='YlGn',
            linewidths=0.5, ax=ax, vmin=0, vmax=1, annot_kws={'size':9})
ax.set_title('Retrieval Metrics Heatmap - All Methods', fontsize=13,
            fontweight='bold')
plt.tight_layout(); plt.show()
```



```
[ ]: # Visualisation 5: Per-Query MRR Breakdown
pivot = eval_df.pivot_table(index='Query', columns='Method', values='MRR').
    ↪fillna(0)
fig, ax = plt.subplots(figsize=(14, 6))
pivot.plot(kind='bar', ax=ax, colormap='tab10', edgecolor='white')
ax.set_title('Per-Query MRR by Retrieval Method', fontsize=13,
    ↪fontweight='bold')
ax.set_xlabel('Query'); ax.set_ylabel('MRR')
ax.set_xticklabels([q[:35]+'...' for q in pivot.index], rotation=40,
    ↪ha='right', fontsize=7)
ax.legend(bbox_to_anchor=(1.01,1), loc='upper left', fontsize=8)
ax.set_ylim(0, 1.1); plt.tight_layout(); plt.show()
```



1.7.1 Evaluation Analysis and Discussion

Key Findings:

Finding	Observation
Best overall	Hybrid-RRF achieves highest MRR and NDCG@5
Dense > BM25	For context-heavy queries (e.g., inflation-equity causal link)
BM25 > Dense	For technical term queries (e.g., price-to-earnings ratio calculation)
L12 > L6	Marginally better quality at ~2x latency cost
TF-IDF < BM25	BM25 length normalisation critical for variable-length financial docs

Retrieval Trade-offs in Financial Systems:

Scenario	Best Method
Regulatory/compliance keyword queries	BM25
Conversational financial advisory	Dense (MiniLM-L6/L12)
Mixed real-world QA	Hybrid-RRF
Low-latency production API	BM25 or Dense-L6
High-precision answer extraction	Hybrid-RRF + L12

Limitations of Retrieval Methods: - Sparse: vocabulary mismatch; cannot handle synonyms or paraphrases. - Dense: slow to index large corpora; requires GPU for low-latency inference. - Hybrid: higher latency than individual methods; adds fusion hyperparameter.

1.8 Design Decisions

1. **Dataset:** FiQA-2018 selected as a dedicated financial QA benchmark covering investment, banking, taxation, and macroeconomics across ~2000 community Q&A answers.
2. **Sparse Retrieval:** BM25 ($k_1=1.5$, $b=0.75$) and TF-IDF (sublinear TF, bigrams, $\min_df=2$). BM25 chosen as primary sparse component due to superior document-length normalisation.
3. **Dense Retrieval:** all-MiniLM-L6-v2 and all-MiniLM-L12-v2 compared. FAISS IndexFlatIP with L2-normalised embeddings used for efficient cosine similarity search.
4. **Hybrid Fusion Strategies:**
 - **RRF ($k=60$):** rank-based, robust, no normalisation needed. Primary strategy.
 - **Weighted Score Fusion ($\alpha=0.4$):** configurable; useful for domain-specific tuning.
5. **Similarity Metrics:** Cosine (L2-normalised FAISS) vs raw dot-product — experimentally verified equivalent with normalised embeddings.
6. **Evaluation:** Pseudo-relevance judgements (BM25 intersect Dense top-15) used as ground truth in absence of explicit FiQA passage-level qrels.

1.9 Challenges Faced

- **No explicit passage-level relevance labels** in FiQA-2018 train split — pseudo-relevance introduces evaluation bias favouring hybrid methods.
- **Finance abbreviation diversity** — abbreviations like ETF, RBI, CAGR required a custom glossary for consistent preprocessing across both sparse and dense pipelines.
- **Latency-quality trade-off** — MiniLM-L12 improved retrieval quality by ~5-8% NDCG over L6, but at 2x inference overhead, requiring careful deployment consideration.
- **Score scale incompatibility** — BM25 scores (0-100+) and cosine scores (0-1) cannot be directly fused without normalisation; RRF elegantly avoids this problem entirely.

1.10 Limitations

- **Pseudo-relevance bias:** Evaluation favours methods that share documents with BM25 and dense — independent human-annotated qrels would give unbiased metrics.

- **No domain-adapted embeddings:** FinBERT or BGE-financial embeddings would likely outperform general-purpose MiniLM on financial domain queries.
- **Template-based generation:** The QA pipeline returns retrieved passages rather than synthesising an abstractive answer — an LLM reader layer would improve response quality.
- **Corpus size:** Limited to 2000 documents for Colab runtime; production deployment would index the full FiQA corpus and beyond.

1.11 Future Improvements

1. **Domain-adapted embeddings:** Use FinBERT or BAAI/bge-base-financial-maturity for higher financial domain alignment.
2. **Cross-encoder re-ranking:** Add a cross-encoder (e.g., cross-encoder/ms-marco-MiniLM-L6-v2) on the hybrid top-20 for precision boosting.
3. **LLM-based answer synthesis:** Use GPT-4o or Claude as an abstractive reader grounded on retrieved passages.
4. **Multi-turn conversation:** Implement query rewriting using conversation history for context-aware follow-up queries.
5. **Production vector DB:** Replace in-memory FAISS with Qdrant or Weaviate for persistent, filterable, multi-tenant vector storage.
6. **Observability:** Integrate Langfuse for retrieval quality monitoring, latency tracking, and A/B testing of retrieval strategies.
7. **Explicit evaluation:** Annotate a gold standard query-passage relevance set for unbiased future evaluation.